DOCKER HOST (no Kubernetes)

Four containers share one Docker engine and one Linux kernel

`web-frontend` · `api-service` · `batch-worker` · `cache-db`

Docker Stats API

deployed capture path

cgroup v2 / eBPF reader

designed, not built

cpu, throttled time, memory, network

40,320 rows across 4 containers (7 days, 1 per minute)

L1 · COLLECTOR

usage + throttled time → one true-demand sample per container

samples, per container, in time order

L2 · FEATURE EXTRACTOR

rolling 10-tick window · stateful, one per container

uses only past samples, so the same code runs offline and live

ONE row · 9 numbers · per container per tick

cpu, mem, net, Δcpu, mean, std, Δmem, time-of-day sin/cos

STAGE 1

memoryless: sees this one row, nothing before it

Isolation Forest

how easy is this row to isolate

from this container's normal rows?

`150 trees · one per container`

Score normaliser

raw score → 0 to 1

burst line = 97th pct of training

`threshold also sent to L3`

OFFLINE TRAINING

days 1-4 train · day 5 validate

days 6-7 held-out test

`fits Stage 1 and Stage 2`

one burst score, appended as the 10th column

not a vote; it becomes input

SLIDING BUFFER

last 30 rows, the oldest drops off as each new one arrives

out ▯ ▯ ▯ ▯ . . . ▯ ▯ ▯ ▯ ▯ **newest** `10 cols`

all 30 rows together, in order

STAGE 2 · LSTM

temporal: reads the sequence, so it sees ramps

30 × 10 → peak CPU and peak memory over the next 3 minutes

32 hidden units · plain NumPy · MSE + Adam · early stopping

two numbers: cores and GB

L3 · ALLOCATOR

CPU target = max(forecast, current use) × (1 + headroom)

headroom 15%; score over the burst line → 15% + 17.5 to 35%

memory floor = 120% of the highest use in the last 3 hours

target above limit → **SCALE UP NOW (+5%)**

below 85% of limit, 5 quiet ticks → **SCALE DOWN** else HOLD

clamped to 0.10-4 cores and 0.128-6 GB per container

L4 · EXECUTOR

`docker update --cpus 2.15`

`--memory 1558m web-frontend`

dry-run by default,

--apply to enforce

OPERATOR PATH

SQLite → graphs → report

demand, forecast, burst score,

limit, reason and command

per container per tick

SWARM / K8S ADAPTER

same policy, other executors

designed, not built

ONLINE RETRAINING

nightly refit on recorded samples

designed, not built

closed loop: new limits take effect on the next tick

data flow · the relay: Stage 1's output is Stage 2's input · feedback · designed but not built